

implementing and analyzing a basic rl algorithm

elsayed elmandouh - 20596379 - cisc 856 assignment 1 - reinforcement learning

1. introduction

this report documents the implementation and analysis of a 5x5 gridworld markov decision process (mdp) solved using three approaches: linear system solving, value iteration, and policy iteration. the goal is to demonstrate mastery of fundamental reinforcement learning concepts including mdp formulation, bellman equations, and dynamic programming algorithms

the gridworld environment presents a classic rl challenge: an agent must navigate a grid while avoiding dangers and reaching a goal. we analyze convergence behavior across different discount factors (0.75, 0.95) and noise levels (0.0 deterministic, 0.2 stochastic)

2. problem definition

2.1 environment parameters

parameter	value
grid size	5x5 (25 cells total)
goal state	cell 14 (center-right) - reward +5.0
danger states	cells 2, 18, 21 - reward -5.0
blocked cells	cells 6, 7, 11, 12 (impassable)
valid states	17 (non-terminal, non-blocked)
actions	up, right, down, left

2.2 mdp components

state space (s): all cells except blocked ones. indexing is row-major: - row 0: cells 0-4 - row 1: cells 5-9 - row 2: cells 10-14 - row 3: cells 15-19 - row 4: cells 20-24

action space (a): four deterministic moves. when hitting a wall or blocked cell, the agent stays in place.

transition model: with probability (1 - noise), the agent moves in the intended direction. with probability noise, the agent moves in a random direction (uniform over the 3 other actions).

reward function: - goal cell (14): +5.0, episode terminates - danger cells (2, 18, 21): -5.0, episode terminates - all other transitions: 0.0

3. implementation

3.1 gridworld class structure

the `gridworld` class (`src/utils/gridworld.py`) implements the full mdp interface with the following key methods:

- `__init__` : initialize environment with all parameters
- `_state_from_action` : compute next state given action (handles walls/blocked)
- `is_terminal` : check if state is goal or danger
- `get_states` : return all valid (non-terminal, non-blocked) states
- `get_actions` : return available actions (all 4 directions)
- `get_reward` : return reward for entering a state
- `get_transitions` : return list of (probability, state) pairs with noise handling
- `solve_linear_system` : solve $ax=b$ for deterministic case

3.2 key implementation details

state indexing: row-major (0-4 in row 0, 5-9 in row 1, etc.)

transition model: stochastic with configurable noise: - deterministic (noise=0.0): always move in intended direction - stochastic (noise=0.2): (1-0.2)=0.8 intended, 0.2/3=0.067 for each other direction

blocked cell handling: agent stays in place when attempting to enter blocked cells.

4. methods

4.1 linear system solver (task ii)

for a given discount factor γ , the state values under uniform random policy satisfy:

$$\mathbf{v} = \mathbf{r} + \gamma \times \mathbf{p} \times \mathbf{v}$$

where: - $\mathbf{v}$: vector of state values (17x1) - $\mathbf{r}$: vector of expected immediate rewards - $\mathbf{p}$: transition probability matrix (17x17) - γ : discount factor

this is rearranged into the linear system form $\mathbf{a}\mathbf{x} = \mathbf{b}$: - $\mathbf{a} = \mathbf{I} - \gamma \times \mathbf{p} / 4$ (for uniform random policy with 4 actions) - $\mathbf{b}$ = expected rewards from $\mathbf{r}$

the system is solved using numpy's `np.linalg.solve()`.

4.2 value iteration (task iii-a)

value iteration directly computes the optimal value function by iteratively applying the bellman optimality operator:

$$\mathbf{v}(\mathbf{s}) \leftarrow \max_a \sum_{\mathbf{s}'} \mathbf{p}(\mathbf{s}'|\mathbf{s},a) [\mathbf{r}(\mathbf{s}') + \gamma \times \mathbf{v}(\mathbf{s}')]]$$

algorithm: 1. initialize $\mathbf{v}(\mathbf{s}) = 0$ for all states 2. repeat until convergence ($\delta < \text{tolerance}$): - create snapshot of current values - for each non-terminal state, compute max over actions - update values and track maximum change (δ) - commit all updates synchronously 3. return number of iterations

4.3 policy iteration (task iii-b)

policy iteration alternates between policy evaluation and policy improvement:

policy evaluation: compute $\mathbf{v}^{\pi}$ for current policy by iterating: $\mathbf{v}(\mathbf{s}) = \sum_{\mathbf{s}'} \mathbf{p}(\mathbf{s}'|\mathbf{s},\pi(\mathbf{s})) [\mathbf{r}(\mathbf{s}') + \gamma \times \mathbf{v}(\mathbf{s}')]]$

policy improvement: update policy greedily: $\pi(\mathbf{s}) = \operatorname{argmax}_a \sum_{\mathbf{s}'} \mathbf{p}(\mathbf{s}'|\mathbf{s},a) [\mathbf{r}(\mathbf{s}') + \gamma \times \mathbf{v}(\mathbf{s}')]]$

algorithm: 1. initialize random policy 2. repeat until policy stable: - policy evaluation: iterate $\mathbf{v}$ under fixed policy - policy improvement: update each state

to best action - check if any policy changes (policy_stable flag) 3. return policy and iteration count

5. results

5.1 linear system solver results

the linear solver produces state values under a **uniform random policy** (not optimal).

discount factor $\gamma = 0.95$:

```
-2.69  -3.60  -5.00  -1.64  -0.00
-2.35  ----  ----  -0.00   1.64
-2.51  ----  ----  -0.00  goal
-3.19  -3.91  -4.10  -5.00  -0.57
-3.82  -5.00  -3.99  -3.45  -1.82
```

discount factor $\gamma = 0.75$:

```
-0.78  -2.24  -5.00  -1.54  -0.00
-0.38  ----  ----  -0.00   1.54
-0.48  ----  ----  -0.00  goal
-1.22  -2.45  -2.73  -5.00  -0.17
-2.37  -5.00  -2.71  -2.34  -0.75
```

analysis: higher discount factor (0.95) propagates goal value further, resulting in higher positive values in nearby states. lower discount (0.75) places more weight on immediate rewards, so states far from the goal have lower values. the negative values near danger cells reflect expected penalty from random wandering.

5.2 value iteration results

configuration	iterations	key observations
$\gamma=0.95$, noise=0.0	12	highest values near goal, clear optimal paths

configuration	iterations	key observations
$\gamma=0.75$, noise=0.0	12	lower values, goal still clearly reachable
$\gamma=0.95$, noise=0.2	15	noise dilutes values, more conservative policy
$\gamma=0.75$, noise=0.2	7	fastest convergence, immediate rewards dominate

$\gamma=0.95$, noise=0.0 (12 iterations):

```

3.15  2.99 -5.00  4.51  4.75
3.32  ----  ----  4.75  5.00
3.49  ----  ----  5.00  goal
3.68  3.87  4.07 -5.00  5.00
3.49 -5.00  4.29  4.51  4.75

```

$\gamma=0.95$, noise=0.2 (15 iterations):

```

0.42 -0.10 -5.00  3.60  4.51
0.56  ----  ----  4.49  4.88
0.65  ----  ----  4.22  goal
0.72  0.83  1.40 -5.00  4.17
0.23 -5.00  2.09  2.90  3.84

```

key observation: noise significantly reduces the magnitude of state values because the agent's movement is less predictable. states near the goal still achieve high values, but intermediate states have more moderate values due to uncertainty about future positions.

5.3 policy iteration results

$\gamma=0.95$, noise=0.0 (10 policy improvement iterations):

```

3.15  2.99 -5.00  4.51  4.75
3.32  ----  ----  4.75  5.00
3.49  ----  ----  5.00  goal

```

3.68	3.87	4.07	-5.00	5.00
3.49	-5.00	4.29	4.51	4.75

the final values match value iteration exactly, confirming correctness of both algorithms.

5.4 optimal policy map ($\gamma=0.95$, noise=0.0)

state	position	optimal action	rationale
0	(0,0)	down	enter grid toward goal
1	(0,1)	left	avoid blocked column
3	(0,3)	right	move toward goal column
4	(0,4)	down	navigate to goal
5	(1,0)	down	enter from top-left
8	(1,3)	right	approach goal
9	(1,4)	down	enter goal column
10	(2,0)	down	continue toward goal
13	(2,3)	right	enter goal cell
15	(3,0)	right	avoid danger at 18
16	(3,1)	right	navigate blocked cells
17	(3,2)	down	go to goal column
19	(3,4)	up	from row 3, reach goal
20	(4,0)	up	enter from bottom-left
22	(4,2)	right	navigate to goal
23	(4,3)	right	approach goal
24	(4,4)	up	enter goal from bottom-right

the optimal policy successfully navigates around blocked cells (6, 7, 11, 12) and avoids danger cells (2, 18, 21) to reach the goal (14).

6. convergence analysis

6.1 iteration counts summary

algorithm	configuration	iterations
linear solver	$\gamma=0.95$, noise=0.0	1 (direct solve)
linear solver	$\gamma=0.75$, noise=0.0	1 (direct solve)
value iteration	$\gamma=0.95$, noise=0.0	12
value iteration	$\gamma=0.75$, noise=0.0	12
value iteration	$\gamma=0.95$, noise=0.2	15
value iteration	$\gamma=0.75$, noise=0.2	7
policy iteration	$\gamma=0.95$, noise=0.0	10 (outer)

6.2 factors affecting convergence

1. discount factor (γ)

the discount factor significantly impacts convergence behavior:

- **higher γ (0.95):** places more weight on future rewards, requiring more iterations to propagate goal value through the grid. the agent considers long-term consequences.
- **lower γ (0.75):** emphasizes immediate rewards, leading to faster stabilization in some cases. the agent focuses on near-term gains.

interestingly, in the stochastic case, $\gamma=0.75$ converged in only 7 iterations while $\gamma=0.95$ required 15 iterations. this counterintuitive result occurs because low discount dampens the effect of transition uncertainty, allowing values to stabilize quickly with conservative estimates.

2. stochasticity (noise)

noise level affects convergence through transition uncertainty:

- **deterministic (noise=0.0)**: clean value propagation. the agent's movement is predictable, allowing efficient backpropagation of goal/danger values.
- **stochastic (noise=0.2)**: increased uncertainty requires more iterations to reach stable estimates. values must account for possible deviation from intended direction.

3. grid topology

the blocked cells (6, 7, 11, 12) create barriers that: - force longer paths from left side of grid to goal - create asymmetric value distributions - require values to propagate around obstacles

the goal at position (2,4) and blocked column at positions 1 and 2 create an interesting navigation challenge that both algorithms solve correctly.

6.3 policy quality comparison

method	policy type	state values	characteristics
linear solver	uniform random	negative near goal	not optimal; reflects random policy
value iteration	optimal	positive near goal	maximizes expected cumulative reward
policy iteration	optimal	positive near goal	identical to vi result

both value iteration and policy iteration converge to the **same optimal policy** for the deterministic case, demonstrating theoretical equivalence.

value quality by method: - linear solver: goal adjacent state (13) = -0.00 (no path optimization) - value iteration: goal adjacent state (13) = 5.00 (optimal path learned) - difference: the linear solver assumes random behavior, while vi/pi find optimal actions.

7. algorithm comparison

7.1 linear system solver

advantages: - direct closed-form solution (single computation) - no iteration required - demonstrates bellman equation as linear system

limitations: - only valid for deterministic environments - only applicable under fixed (random) policy - cannot compute optimal policy

when to use: baseline validation, small deterministic mdps.

7.2 value iteration

advantages: - works for stochastic and deterministic environments - directly computes optimal value function - simple, straightforward implementation

limitations: - may require many iterations for tight tolerance - computes max over all actions at each update - no policy information until algorithm completes

when to use: general-purpose optimal control.

7.3 policy iteration

advantages: - often fewer outer iterations than vi - explicit policy at each improvement step - can leverage early stopping if policy stabilizes

limitations: - policy evaluation requires inner iteration loop - higher per-iteration cost - more complex implementation

when to use: problems where policy converges quickly, or when intermediate policies are useful.

8. visual results

generated visualization files in docs/02-results/ : -

linear_solver_gamma0.95_noise0.0.png - linear solver values for $\gamma=0.95$ -

linear_solver_gamma0.75_noise0.0.png - linear solver values for $\gamma=0.75$ -

vi_gamma0.95_noise0.0.png - value iteration for $\gamma=0.95$, deterministic -

vi_gamma0.95_noise0.2.png - value iteration for $\gamma=0.95$, stochastic -

vi_gamma0.75_noise0.0.png - value iteration for $\gamma=0.75$, deterministic -

vi_gamma0.75_noise0.2.png - value iteration for $\gamma=0.75$, stochastic -

pi_gamma0.95_noise0.0.png - policy iteration for $\gamma=0.95$ -
optimal_policy_gamma0.95.png - optimal policy arrows